

C# / .NET Learning Roadmap

Topic Notes — Section 2: Object-Oriented Programming in C#

Topics: 2.1 through 2.17

Date: 29 June 2026

Section 2 — Object-Oriented Programming in C#

This section covers OOP in C# — the principles and language features that structure every .NET application. Topics build on the C# fundamentals from Section 1.

2.1. Classes and Objects

Builds on: 1.4 (reference types — classes are reference types), 1.11 (methods — classes contain them), 1.16 (structs vs classes — you already know the contrast), 1.19 (scope — class fields have class-level scope).

What is a class and why does it exist?

A class is a blueprint. An object is what you create from that blueprint.

Without classes, you'd have loose variables and methods floating around with no relationship to each other. A class groups related data (fields/properties) and behaviour (methods) into a single named unit — the fundamental building block of object-oriented programming.

```
// Blueprint:
public class Order
{
    public int Id { get; set; }
    public string CustomerName { get; set; }
    public decimal Amount { get; set; }

    public bool IsHighValue() => Amount > 10000;
}

// Objects -- instances of the blueprint:
var order1 = new Order { Id = 1, CustomerName = "Sathish", Amount = 15000 };
var order2 = new Order { Id = 2, CustomerName = "Priya", Amount = 500 };

Console.WriteLine(order1.IsHighValue()); // true
Console.WriteLine(order2.IsHighValue()); // false
```

order1 and order2 are independent objects. Same blueprint, different data.

Anatomy of a class

```
public class BankAccount
{
    private decimal _balance;
    public string AccountNumber { get; private set; }
    public decimal Balance => _balance;

    public BankAccount(string accountNumber, decimal initialBalance)
    {
        AccountNumber = accountNumber;
        _balance = initialBalance;
    }

    public void Deposit(decimal amount)
    {
```

```

        if (amount <= 0) throw new ArgumentException("Amount must be positive");
        _balance += amount;
    }

    public bool Withdraw(decimal amount)
    {
        if (amount > _balance) return false;
        _balance -= amount;
        return true;
    }

    public override string ToString()
        => $"Account {AccountNumber}: {_balance:C}";
}

```

Fields vs Properties

```

// Fields -- raw storage (always private):
private decimal _balance;
private readonly ILogger _logger;

// Properties -- controlled access:
public string Name { get; set; }           // auto-property
public string Id { get; }                  // read-only
public string Status { get; private set; } // private setter
public decimal TotalWithTax => Amount * 1.18m; // computed
public string Reference { get; init; }     // init-only

```

In practice: fields are private, properties are public. Never expose raw fields publicly.

Object initialiser syntax

```

var order = new Order
{
    Id = 1,
    CustomerName = "Sathish",
    Amount = 999.99m
};

```

Access modifiers on class members

Modifier	Accessible from
public	Anywhere
private	This class only
protected	This class and subclasses
internal	This assembly (project) only
private protected	This class and subclasses in same assembly
protected internal	This assembly OR subclasses anywhere

Static class members

```

public class Order
{
    private static int _nextId = 1;
}

```

```

    public int Id { get; }

    public Order() { Id = _nextId++; }

    public static Order CreateEmpty() => new Order();
}

var a = new Order();           // Id = 1
var b = new Order();           // Id = 2
var c = Order.CreateEmpty();   // Id = 3

```

Realistic scenario — a complete small class

```

public class PasswordValidator
{
    private readonly int _minLength;
    private readonly bool _requireSpecialChar;

    public PasswordValidator(int minLength = 8, bool requireSpecialChar = true)
    {
        _minLength = minLength;
        _requireSpecialChar = requireSpecialChar;
    }

    public (bool isValid, string error) Validate(string password)
    {
        if (string.IsNullOrEmpty(password))
            return (false, "Password is required");
        if (password.Length < _minLength)
            return (false, $"Must be at least {_minLength} characters");
        if (_requireSpecialChar && !password.Any(c => !char.IsLetterOrDigit(c)))
            return (false, "Must contain a special character");
        return (true, string.Empty);
    }
}

```

Key Takeaways

- A class is a blueprint defining data (fields/properties) and behaviour (methods)
- An object is an instance created from a class with `new`
- Fields are private raw storage; properties are controlled public access
- Static members belong to the type and are shared; instance members belong to each object

Member	Purpose	Typical access
Field	Raw storage	private
Property	Controlled access to data	public
Constructor	Initialise the object	public
Method	Behaviour / operations	public or private
Static	Shared across instances	public static

.NET-specific rules:

- Properties over public fields — always

- Prefix private fields with `_`
- `{ get; init; }` for immutable-after-creation properties
- override `ToString()` when the class will appear in logs or debug output

Memory aid: Class = blueprint, Object = house built from it. One blueprint, many independent houses.

Debugging Tips

Symptom	Cause	Fix
<code>NullReferenceException</code> on a property	Object was never initialised with <code>new</code>	Check if the variable is null before use; ensure <code>new</code> was called
Property change not reflected elsewhere	May have copied the object initialiser to a new variable — check if you wanted a new object or the same reference	Verify whether you wanted a new object or the same one
Cannot access private member	Accessing private field/property from outside the class	Expose through a public property or method
Object initialiser won't compile	Property has no setter or only private set	Add set, change to <code>init</code> , or use a constructor parameter
Static member gives stale/shared data unexpectedly	Forgot it's shared — one request modified it for all	Move shared mutable state to instance members or use DI scoping

.NET-specific: In ASP.NET Core, services registered as `Singleton` behave like static — one instance shared across all requests. Use `Scoped` for per-request state.

Self-check — you should now be able to:

- Define a class with private fields, public properties, a constructor, and methods
- Explain the difference between a class (blueprint) and an object (instance)
- Choose between `{ get; set; }`, `{ get; }`, `{ get; private set; }`, and `{ get; init; }` correctly
- Explain what static means on a class member and when to use it

2.2. Constructors (Default, Parameterized, Static, Primary Constructors)

Builds on: 2.1 (classes — constructors are what run when you call `new`), 1.12 (optional parameters), 1.18 (readonly — fields set in constructors are the primary readonly pattern).

What are constructors and why do they exist?

Without a constructor, there's no controlled way to ensure an object starts in a valid state. A constructor is a special method that runs automatically when you create an object with `new`. Its job is to initialise the object into a valid, usable state from the first moment it exists.

Default Constructor

```
public class Order { public int Id { get; set; } public string Status { get; set; } }
```

```
// Compiler generates invisibly: public Order() { }
var order = new Order();    // works

// Once you define ANY constructor, compiler stops generating the default.
// If you still want parameterless, write it explicitly.
```

Parameterised Constructor

```
public class Order
{
    public int Id { get; }
    public string CustomerName { get; }
    public decimal Amount { get; }
    public string Status { get; private set; }

    public Order(int id, string customerName, decimal amount)
    {
        if (string.IsNullOrEmpty(customerName))
            throw new ArgumentException("Customer name is required");
        if (amount <= 0)
            throw new ArgumentException("Amount must be positive");

        Id = id;
        CustomerName = customerName;
        Amount = amount;
        Status = "Pending";
    }
}

var order = new Order(1, "Sathish", 999.99m);    // guaranteed valid
```

Constructor Overloading and Chaining

```
public class EmailMessage
{
    public string To { get; }
    public string Subject { get; }
    public string Body { get; }
    public bool IsHtml { get; }

    // Convenience constructor delegates to full constructor:
    public EmailMessage(string to, string subject)
        : this(to, subject, string.Empty, false) { }

    public EmailMessage(string to, string subject, string body, bool isHtml)
    {
        To = to; Subject = subject; Body = body; IsHtml = isHtml;
    }
}
```

The `: this(...)` syntax is constructor chaining. The chained constructor runs first, then the calling constructor's body runs.

Constructor Chaining with `: base()`

```

public class Entity
{
    public int Id { get; }
    public DateTime CreatedAt { get; }
    public Entity(int id) { Id = id; CreatedAt = DateTime.UtcNow; }
}

public class Order : Entity
{
    public string CustomerName { get; }
    public Order(int id, string customerName) : base(id)    // calls Entity(int id) first
        => CustomerName = customerName;
}

```

Static Constructor

```

public class AppConfig
{
    public static readonly string Environment;
    public static readonly bool IsProduction;

    static AppConfig()    // No parameters, no access modifier, runs once:
    {
        Environment = System.Environment
            .GetEnvironmentVariable("ASPNETCORE_ENVIRONMENT") ?? "Development";
        IsProduction = Environment == "Production";
    }
}

```

Primary Constructors — C# 12

```

// Traditional:
public class OrderService
{
    private readonly IOrderRepository _repo;
    private readonly ILogger<OrderService> _logger;
    public OrderService(IOrderRepository repo, ILogger<OrderService> logger)
    { _repo = repo; _logger = logger; }
}

// With primary constructor -- capture into readonly fields for robustness:
public class OrderService(IOrderRepository repo, ILogger<OrderService> logger)
{
    private readonly IOrderRepository _repo = repo;
    private readonly ILogger<OrderService> _logger = logger;

    public async Task<Order?> GetByIdAsync(int id)
    {
        _logger.LogInformation("Getting order {Id}", id);
        return await _repo.GetByIdAsync(id);
    }
}

```

DI Constructor Pattern in ASP.NET Core

```

[ApiController]
[Route("api/orders")]
public class OrdersController : ControllerBase
{
    private readonly IOrderService _orderService;
    private readonly ILogger<OrdersController> _logger;

    public OrdersController(IOrderService orderService, ILogger<OrdersController> logger)
    {
        _orderService = orderService;
        _logger = logger;
    }
}

```

Key Takeaways

- A constructor runs automatically when new is called
- If you define any constructor, the compiler stops generating the default parameterless one
- Constructor chaining (: this(...)) lets one constructor call another to avoid duplicating logic
- Static constructors run once before first use, have no parameters and no access modifier
- Primary constructors (C# 12) — capture into readonly fields for robustness

Constructor type	When it runs	Parameters
Default (generated)	new ClassName()	None
Parameterised	new ClassName(args)	Yes
Chained (: this)	Inside another ctor	Varies
Base (: base)	Inside derived ctor	Varies
Static	Before first class use	None
Primary (C# 12)	new ClassName(args)	On decl

.NET-specific rules:

- Always validate constructor parameters — throw ArgumentException or ArgumentNullException
- Use ArgumentNullException.ThrowIfNull(param) (.NET 6+) for clean null checks
- EF Core requires a parameterless constructor on entity classes (can be private or protected)

Memory aid: Constructor = the object's birth certificate. It must be filled in correctly before the object enters the world.

Debugging Tips

Symptom	Cause	Fix
No parameterless constructor defined	You added a parameterised constructor and the framework tries to instantiate with new()	Add a public ClassName() {} or configure the framework to use a different constructor
DI throws Unable to resolve service	Constructor parameter type not registered in DI container	Add the missing registration in Program.cs
Object reference not set immediately after new	A required property wasn't set — object initialiser missed a property	Use a constructor that requires the value

Symptom	Cause	Fix
Static constructor runs multiple times	It doesn't — you're likely confusing it with an instance constructor	Static constructors run exactly once per app domain
Primary constructor parameter is null at runtime	DI failed to resolve or you passed null explicitly	Validate with <code>ArgumentNullException.ThrowIfNull()</code> at start of constructor body

.NET-specific: EF Core requires a parameterless constructor on entity classes (can be private) for materialising query results.

Self-check — you should now be able to:

- Write a class with a parameterised constructor that validates its inputs
- Explain what happens when you define a constructor and why the default disappears
- Use constructor chaining (`: this(...)`) to avoid duplicated initialisation logic
- Write the standard DI constructor pattern used in every ASP.NET Core service and controller

2.3. Properties (get/set, Auto-Properties, init-only)

Builds on: 2.1 (classes — properties are class members), 2.2 (constructors — init-only properties are set there), 1.18 (readonly — init-only is the property equivalent).

What are properties and why do they exist?

A public field gives direct, uncontrolled access to data — anyone can set it to an invalid value. Properties wrap data access in getter and setter methods with cleaner syntax, enabling validation and controlled access.

```
public class Order
{
    private decimal _amount;

    public decimal Amount
    {
        get { return _amount; }
        set
        {
            if (value <= 0) throw new ArgumentException("Amount must be positive");
            _amount = value;
        }
    }
}

order.Amount = -500; // throws ArgumentException
order.Amount = 999; // fine
```

Auto-Properties — the shorthand

```
// Full manual property:
```

```
private string _name;
public string Name { get { return _name; } set { _name = value; } }

// Auto-property -- identical behaviour, compiler generates the field:
public string Name { get; set; }
```

Property access modifier variants

```
public string Status { get; set; }           // Anyone can read and write
public string Status { get; private set; }  // Externally read-only
public string Status { get; protected set; } // Subclasses can write
public int Id { get; }                      // Set only in constructor
public string Reference { get; init; }       // Set in initialiser only
```

init-only properties — C# 9

```
public class Order
{
    public int Id { get; init; }
    public string CustomerName { get; init; }
    public decimal Amount { get; init; }
}

// Object initialiser -- allowed:
var order = new Order { Id = 1, CustomerName = "Sathish", Amount = 999.99m };

// After construction -- not allowed:
order.Id = 2;    // compile error -- init-only
```

{ get; } requires a constructor. { get; init; } works with object initialisers too — more flexible for DTOs.

Computed properties

```
public class Order
{
    public decimal Subtotal { get; set; }
    public decimal TaxRate { get; set; }

    public decimal Tax => Subtotal * TaxRate;           // computed on access
    public decimal Total => Subtotal + Tax;             // recalculates each time
    public bool IsHighValue => Total > 10000;
}
```

Required properties — C# 11

```
public class CreateOrderDto
{
    public required string CustomerName { get; set; }
    public required decimal Amount { get; set; }
    public string? Notes { get; set; }                // optional
}

var dto = new CreateOrderDto();    // compile error -- required properties not set
var dto = new CreateOrderDto { CustomerName = "Sathish", Amount = 999.99m };    // fine
```

Properties in EF Core

```
public class Product
{
    public int Id { get; set; } // primary key
    public string Name { get; set; } // NOT NULL column
    public string? Description { get; set; } // nullable column
    public decimal Price { get; set; } // NOT NULL column
    public bool IsActive { get; set; } = true; // default value
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
}
```

Key Takeaways

- Properties control access to data — they look like fields but behave like methods
- Auto-properties ({ get; set; }) are the default — add a backing field only when you need custom logic
- { get; } — read-only, set only in constructor
- { get; init; } — read-only after construction, settable in object initialisers
- { get; private set; } — externally readable, only the class can write
- Computed properties (=> expression) derive from other data, recalculate on each access

Syntax	Can read	Can write	When writable
{ get; set; }	Anyone	Anyone	Anytime
{ get; private set; }	Anyone	This class	Anytime (in class)
{ get; protected set; }	Anyone	Class+sub	Anytime (hierarchy)
{ get; }	Anyone	This class	Constructor only
{ get; init; }	Anyone	Anyone	Initialisation only
=> expression	Anyone	N/A	Read-only, computed

Memory aid: Property = a security guard for your data. The guard decides who gets in and checks credentials before letting anyone change anything.

Debugging Tips

Symptom	Cause	Fix
EF Core skips property in query results	Property has no setter or private set — EF can't materialise it	Add set; or use EF's HasField configuration
init property can't be set in a method	init-only allows setting only during initialisation	Use private set if post-construction mutation is needed
Model binding doesn't populate property	Property has no setter	Add { get; set; } or { get; init; }
Computed property returns stale value	You're caching the result in a variable	Access the property directly each time — it recalculates on access
required property error at runtime	Missing required property in object initialiser	Set all required properties or remove the required modifier if optional

.NET-specific: JSON deserialization (System.Text.Json) requires either a set or init accessor. { get; } only properties will be silently skipped during deserialization.

Self-check — you should now be able to:

- Choose the right property syntax for a given scenario from the six variants
- Write a property with custom validation logic in the setter
- Distinguish between { get; } and { get; init; } and when each is appropriate
- Explain why EF Core entities need { get; set; } and why DTOs can use { get; init; }

2.4. Access Modifiers (public, private, protected, internal, etc.)

Builds on: 2.1 (classes), 2.3 (properties — you've already used private set and protected set), 1.2 (project structure — internal is scoped to a project/assembly).

What are access modifiers and why do they exist?

Without access modifiers, every piece of code could read and modify every field, method, and property of every class. Access modifiers enforce encapsulation — the principle that a class controls what it exposes and hides its implementation details.

```
public           -- accessible from anywhere
private          -- accessible only within this class
protected        -- accessible within this class and any subclass
internal         -- accessible within this assembly (project) only
protected internal -- accessible within this assembly OR any subclass
private protected -- accessible within this class and subclasses in same assembly
```

Default access modifiers

```
class Order { }           // internal
public class Order { }    // public

public class Order
{
    int _id;               // private
    void Process() { }     // private
}
```

public, private, internal, protected examples

```
// public -- accessible from anywhere:
public class OrderService
{
    public Order GetById(int id) { ... }
}

// private -- this class only:
public class OrderService
{
    private readonly IOrderRepository _repo;
    private void LogAccess(int id) { ... } // internal helper
}

// internal -- this project only:
```

```
internal class SqlOrderRepository : IOrderRepository { }

// protected -- base class shared with subclasses:
public abstract class BaseController : ControllerBase
{
    protected ILogger Logger { get; }
    protected IActionResult HandleNotFound(string resource)
        => NotFound(new { error = $"{resource} not found" });
}
```

InternalsVisibleTo

```
[assembly: InternalsVisibleTo("MyApp.Tests")]
```

Now MyApp.Tests can see internal members of MyApp — standard for testing internal classes.

Key Takeaways

- Start everything private. Open up to internal when another class in the same project needs it. Open up to public only when it forms part of the intended API surface

Modifier	Same class	Same assembly	Subclass	Anyone
private	Yes	No	No	No
private protected	Yes	Yes(sub only)	No	No
protected	Yes	No	Yes	No
internal	Yes	Yes	No	No
protected internal	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

.NET-specific rules:

- Default for types: internal; Default for members: private
- ASP.NET Core controllers and action methods must be public
- EF Core can use private setters on entity properties — it uses reflection

Memory aid: Access modifiers are like doors in a building. private = your office. internal = your floor. protected = your floor plus branches of your department. public = the lobby.

Debugging Tips

Symptom	Cause	Fix
inaccessible due to its protection level	Trying to access a private or internal member from outside its scope	Check the modifier — expose via a public method if intentional access is needed
Test project can't access internal class	Tests are a separate assembly — internal is invisible	Add [assembly: InternalsVisibleTo("YourTests")] to the main project
Controller action returns 404 for all routes	Action method is not public	Make the action method public
EF Core doesn't populate a property	Property has no setter (only { get; })	Add private set; — EF Core can use it via reflection

Symptom	Cause	Fix
Interface implementation gives compile error	Interface member declared with access modifier	Remove the modifier — interface members are implicitly public

Self-check — you should now be able to:

- Name all six access modifiers and describe what each allows
- Apply the 'start private, open up only when needed' principle
- Explain why internal is appropriate for concrete infrastructure implementations

2.5. Encapsulation

Builds on: 2.1 (classes — encapsulation is the core principle behind why classes exist), 2.3 (properties), 2.4 (access modifiers — the enforcement tool).

What is encapsulation and why does it exist?

Encapsulation is the principle of bundling data and the behaviour that operates on it together, while hiding internal implementation details. When external code can directly manipulate internal state, that object can be put into invalid states — and the object has no way to prevent it.

```
// No encapsulation:
public class BankAccount { public decimal Balance; public bool IsActive; }
account.Balance = -99999;    // invalid

// Encapsulated:
public class BankAccount
{
    private decimal _balance;
    private bool _isActive = true;

    public decimal Balance => _balance;
    public bool IsActive => _isActive;

    public void Deposit(decimal amount)
    {
        if (!_isActive) throw new InvalidOperationException("Account is closed");
        if (amount <= 0) throw new ArgumentException("Amount must be positive");
        _balance += amount;
    }

    public void Close() { _isActive = false; }
}
```

Tell, don't ask

```
// BAD -- asking pattern:
if (order.Status == "Pending" && order.Amount > 0)
    order.Status = "Processing";    // external code modifying state

// GOOD -- telling pattern:
```

```

order.StartProcessing(); // Order enforces its own rules

public void StartProcessing()
{
    if (Status != "Pending")
        throw new InvalidOperationException($"Cannot process with status {Status}");
    Status = "Processing";
    ProcessingStartedAt = DateTime.UtcNow;
}

```

Encapsulation with collections

```

// GOOD -- collection is protected:
public class Order
{
    private readonly List<OrderItem> _items = new();

    public IReadOnlyList<OrderItem> Items => _items.AsReadOnly();
    public decimal Total => _items.Sum(i => i.Price * i.Quantity);

    public void AddItem(OrderItem item)
    {
        if (item == null) throw new ArgumentNullException(nameof(item));
        if (_items.Any(i => i.ProductId == item.ProductId))
            throw new InvalidOperationException("Product already in order");
        _items.Add(item);
    }
}

```

Key Takeaways

- Encapsulation = bundling data + behaviour together, hiding internal details
- The object is responsible for keeping itself in a valid state
- Three tools: private fields, controlled properties, public methods as state-change entry points
- Tell, don't ask — tell objects to do things rather than reading their state externally
- Expose collections as IReadOnlyList — never as List

Memory aid: Encapsulation is like a vending machine. You press buttons (public methods), you get the result. You never reach inside to grab the mechanism directly.

Debugging Tips

Symptom	Cause	Fix
Object ends up in invalid state	State is being set directly via public fields or properties with public setters	Make fields private, remove public setters, add validation in methods
IReadOnlyList throws at runtime when caller tries to add	Caller is casting to List to bypass the restriction	This is intentional — callers should use the provided add/remove methods
EF Core can't load related entities	Navigation property exposed as IReadOnlyList without backing field config	Configure EF to use the private backing field with HasField in OnModelCreating

Symptom	Cause	Fix
Business rule violated despite validation in method	External code is setting properties directly, bypassing the method	Remove public set — use private set and route all changes through methods
Unit test can't access internal state to assert	Private fields are hidden	Test behaviour through public methods and return values, not internal state

Self-check — you should now be able to:

- Explain what encapsulation is and the problem it solves in one sentence
- Apply the 'tell, don't ask' principle to replace direct property access with method calls
- Expose a collection safely using IReadOnlyList and AsReadOnly()

2.6. Inheritance

Builds on: 2.1 (classes), 2.2 (constructors — base constructors run first), 2.4 (access modifiers — protected exists specifically for inheritance), 2.5 (encapsulation — inheritance can break it if misused).

What is inheritance and why does it exist?

Inheritance lets a class (the derived class) acquire the members of another class (the base class), extending or specialising its behaviour without rewriting it. When multiple classes share the same data and behaviour, inheritance centralises shared code in one place.

```
public class Payment
{
    public decimal Amount { get; }
    public Payment(decimal amount) { Amount = amount; }
    public virtual string GetDescription() => $"Payment of {Amount:C}";
    public void LogPayment() => Console.WriteLine(GetDescription());
}

public class CreditCardPayment : Payment
{
    public string LastFour { get; }
    public CreditCardPayment(decimal amount, string lastFour) : base(amount)
        => LastFour = lastFour;

    public override string GetDescription()
        => $"Credit card payment of {Amount:C} (card ending {LastFour})";
}
```

base keyword

```
public class CreditCardPayment : Payment
{
    public override string GetDescription()
    {
        var baseDescription = base.GetDescription();
        return $"{baseDescription} via credit card ending {LastFour}";
    }
}
```



```
}
```

Custom exception hierarchy

```
public class AppException : Exception
{
    public int StatusCode { get; }
    public AppException(string message, int statusCode = 400)
        : base(message) => StatusCode = statusCode;
}

public class NotFoundException : AppException
{
    public NotFoundException(string resource)
        : base($"{resource} was not found", statusCode: 404) { }
}
```

BaseEntity pattern

```
public abstract class BaseEntity
{
    public int Id { get; private set; }
    public DateTime CreatedAt { get; private set; } = DateTime.UtcNow;
    public DateTime? UpdatedAt { get; private set; }
    public string CreatedBy { get; private set; }

    protected BaseEntity() { }
    protected BaseEntity(string createdBy) { CreatedBy = createdBy; }
    public void SetUpdated() { UpdatedAt = DateTime.UtcNow; }
}

public class Order : BaseEntity
{
    public string CustomerName { get; private set; }
    public decimal Amount { get; private set; }

    protected Order() { }
    public Order(string customerName, decimal amount, string createdBy)
        : base(createdBy) { CustomerName = customerName; Amount = amount; }
}
```

Key Takeaways

- Inheritance lets a derived class acquire members of a base class — is-a relationship
- C# supports single inheritance only — one direct base class
- virtual + override enable polymorphic behaviour
- Prefer composition over inheritance for behaviour reuse — use inheritance for genuine is-a hierarchies

Keyword	Meaning
: BaseClass	Inherit from BaseClass
: base(args)	Call base class constructor
virtual	Derived classes may override

```

override      | Replaces base implementation
sealed        | Prevents further subclassing/overriding
base.Method() | Call the base class version

```

Memory aid: Inheritance = genetic traits. A derived class inherits its parent's DNA and can express them differently. But you only have one biological parent (single inheritance).

Debugging Tips

Symptom	Cause	Fix
Base method runs instead of override	Method not marked virtual in base class	Add virtual to base method, override to derived
override keyword compile error	Base method isn't virtual or abstract	Mark the base method virtual or use new (hiding — different thing)
EF Core throws on entity materialisation	Entity lacks parameterless constructor	Add protected EntityName() {}
Constructor in derived class doesn't compile	Base class has no matching constructor for : base(...)	Add the required constructor to the base class or adjust the call
sealed class being subclassed	Trying to inherit from a sealed class	Remove sealed or use composition instead

Self-check — you should now be able to:

- Write a base class and two derived classes that inherit from it using : BaseClass
- Explain the difference between virtual/override and a non-virtual method
- Apply the is-a test to decide whether inheritance is appropriate

2.7. Polymorphism (virtual, override)

Builds on: 2.6 (inheritance — polymorphism requires an inheritance relationship), 2.4 (access modifiers), 2.8 (abstract classes — previewed here).

What is polymorphism and why does it exist?

Polymorphism means 'many forms.' The same code can work with objects of different types, and each type responds in its own way.

```

// Without polymorphism -- fragile:
public void ProcessPayment(object payment)
{
    if (payment is CreditCardPayment cc) ChargeCard(cc.CardNumber, cc.Amount);
    else if (payment is BankTransferPayment bt) InitiateTransfer(bt.AccountNumber, bt.Amount);
    // Every new payment type requires modifying this method
}

// With polymorphism -- stable, extensible:
public void ProcessPayment(Payment payment)
{
    payment.Execute();    // each type knows how to execute itself
}

```

Runtime dispatch

```
Payment[] payments = {
    new CreditCardPayment(100m, "4242"),
    new BankTransferPayment(250m, "ACC-001"),
    new Payment(500m)
};

foreach (Payment p in payments)
    p.Execute();

// Output:
// Charging card ending 4242 for $100.00
// Transferring $250.00 to account ACC-001
// Processing payment of $500.00
```

The variable is typed as `Payment`, but the actual object type determines which `Execute()` runs. This is runtime polymorphism.

Polymorphism with collections — Strategy pattern

```
public abstract class DiscountStrategy
{
    public abstract decimal Calculate(decimal orderAmount);
}

public class PercentageDiscount : DiscountStrategy
{
    private readonly decimal _percent;
    public PercentageDiscount(decimal percent) => _percent = percent;
    public override decimal Calculate(decimal orderAmount)
        => orderAmount * (_percent / 100);
}

// Pricing service works with ANY discount strategy:
public class PricingService
{
    public decimal ApplyDiscount(decimal amount, DiscountStrategy discount)
        => amount - discount.Calculate(amount);
}
```

Key Takeaways

- Polymorphism = same code, different behaviour depending on the actual runtime type
- The runtime looks at the actual object type, not the declared variable type
- Enables code that works with current and future types without modification
- `base.Method()` extends rather than replaces base behaviour

Keyword	Meaning
virtual	Can be overridden by derived classes
override	Replaces (or extends) the virtual method
sealed override	Overrides and prevents further overriding
base.Method()	Calls the base class version

`abstract` | Must be overridden -- no default impl

Memory aid: Polymorphism is like a remote control. The same Play button works on a DVD player, streaming box, and Blu-ray player — each responds in its own way.

Debugging Tips

Symptom	Cause	Fix
Base class method runs instead of override	Method not marked virtual in base class	Add virtual to base method
Override not found at runtime	Object was actually the base type, not the derived type	Check where the object was created — <code>new BaseClass()</code> vs <code>new DerivedClass()</code>
<code>new</code> keyword used instead of <code>override</code>	Method hiding (section 2.16), not polymorphism	Replace <code>new</code> with <code>override</code> and make base method virtual
Type check chain getting long	Missing virtual method on base class that should be polymorphic	Move the behaviour into a virtual method on the base class
<code>base.Method()</code> causes stack overflow	Circular delegation	Check for circular calls — ensure <code>base.</code> resolves to actual base

Self-check — you should now be able to:

- Explain what runtime polymorphism is and how `virtual/override` enables it
- Demonstrate that the runtime uses the actual object type, not the variable's declared type
- Apply polymorphism to write code that handles current and future subtypes without modification

2.8. Abstract Classes

Builds on: 2.6 (inheritance — abstract classes are inherited), 2.7 (polymorphism — abstract methods are the strongest form of virtual), 2.4 (access modifiers).

What are abstract classes and why do they exist?

Virtual methods provide a default implementation that derived classes can override. But sometimes there is no sensible default — the base class knows a method must exist, but has no idea how to implement it. Abstract classes define a contract ('you must implement these methods') while still providing shared state and behaviour.

```
public abstract class TaxCalculator
{
    public string Region { get; }
    public decimal TaxRate { get; protected set; }

    protected TaxCalculator(string region, decimal taxRate)
    { Region = region; TaxRate = taxRate; }

    public abstract decimal Calculate(decimal amount);    // MUST be implemented
    public abstract string TaxName { get; }

    public string GetSummary(decimal amount)              // shared by all
}
```

```

        => $"{TaxName} ({Region}): {Calculate(amount):C} on {amount:C}";
    }

    // Cannot instantiate:
    var calc = new TaxCalculator("India", 0.18m);    // compile error

    public class GstCalculator : TaxCalculator
    {
        public GstCalculator() : base("India", 0.18m) { }
        public override decimal Calculate(decimal amount) => amount * TaxRate;
        public override string TaxName => "GST";
    }

```

Template Method pattern

```

public abstract class ReportGenerator
{
    // Sealed -- sequence enforced, steps overridable:
    public sealed Report Generate(ReportRequest request)
    {
        var data = FetchData(request);
        var processed = ProcessData(data);
        var formatted = FormatOutput(processed);
        AddHeader(formatted, request);
        AddFooter(formatted);
        return new Report(formatted);
    }

    protected abstract IEnumerable<object> FetchData(ReportRequest request);
    protected abstract IEnumerable<object> ProcessData(IEnumerable<object> data);
    protected abstract string FormatOutput(IEnumerable<object> data);

    private void AddHeader(string content, ReportRequest request)
        => Console.WriteLine($"=== {request.Title} ===");
    private void AddFooter(string content)
        => Console.WriteLine($"Generated: {DateTime.UtcNow}");
}

```

BackgroundService in ASP.NET Core

```

// ASP.NET Core provides BackgroundService as an abstract class:
public class EmailQueueProcessor : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            await ProcessNextEmailAsync();
            await Task.Delay(1000, stoppingToken);
        }
    }
}

```

Key Takeaways

- An abstract class cannot be instantiated — only its concrete subclasses can
- Abstract methods have no implementation — derived classes must override them
- Concrete methods in abstract classes are shared by all derived classes
- The Template Method pattern — define algorithm skeleton in base, fill steps in derived
- Use interface when you only need a contract with no shared state

Feature	abstract method	virtual method	Interface method
Has implementation	No	Yes (default)	No (usually)
Must override	Yes	No	Yes
Class instantiated	No	Yes	N/A
Can have fields	Yes	Yes	No
Multiple per class	No	No	Yes

Memory aid: An abstract class is an unfinished blueprint. The architect drew the floor plan but left blank spaces. You cannot live in the blueprint itself — only in a finished unit.

Debugging Tips

Symptom	Cause	Fix
Cannot create an instance of the abstract class	Trying to new an abstract class directly	Instantiate a concrete derived class instead
Derived class won't compile	Not all abstract members implemented	Implement every abstract method and property, or mark the derived class abstract too
Abstract method accidentally has a body	abstract methods cannot have implementation	Remove the body — abstract methods end with ;
Template method calls abstract but wrong version runs	Virtual dispatch working correctly — check which concrete type is instantiated	Verify which derived class is being created with GetType().Name
EF Core can't materialise abstract entity	Trying to query an abstract base type directly	Query the concrete type: <code>_context.Set()</code>

Self-check — you should now be able to:

- Explain the difference between abstract and virtual and when to choose each
- Write an abstract class with abstract methods, concrete methods, and shared state
- Apply the Template Method pattern
- Decide between abstract class and interface given a design scenario

2.9. Interfaces (including Default Interface Methods)

Builds on: 2.7 (polymorphism — interfaces are the other polymorphism mechanism), 2.8 (abstract classes — interface vs abstract class is a core design decision), 2.4 (access modifiers — interface members are implicitly public).

What are interfaces and why do they exist?

An interface defines a pure contract — just a set of capabilities, with no state and no implementation. A class can implement many interfaces but inherit only one class. This lets you define multiple independent capabilities and mix them freely. Interfaces also decouple code from implementations — the foundation of dependency injection.

```
public interface IOrderRepository
{
    Task<Order?> GetByIdAsync(int id);
    Task<IEnumerable<Order>> GetAllAsync();
    Task<Order> CreateAsync(Order order);
    Task UpdateAsync(Order order);
    Task DeleteAsync(int id);
}
```

Implementing an interface

```
public class SqlOrderRepository : IOrderRepository
{
    private readonly AppDbContext _context;
    public SqlOrderRepository(AppDbContext context) { _context = context; }

    public async Task<Order?> GetByIdAsync(int id)
        => await _context.Orders.FindAsync(id);

    public async Task<IEnumerable<Order>> GetAllAsync()
        => await _context.Orders.ToListAsync();

    public async Task<Order> CreateAsync(Order order)
    {
        _context.Orders.Add(order);
        await _context.SaveChangesAsync();
        return order;
    }

    public async Task UpdateAsync(Order order)
    { _context.Orders.Update(order); await _context.SaveChangesAsync(); }

    public async Task DeleteAsync(int id)
    {
        var order = await GetByIdAsync(id);
        if (order != null) { _context.Orders.Remove(order); await _context.SaveChangesAsync(); }
    }
}
```

Default interface methods — C# 8+

```
public interface IOrderRepository
{
    Task<Order?> GetByIdAsync(int id);

    // Default implementation -- existing classes don't need to implement:
    async Task<bool> ExistsAsync(int id)
        => await GetByIdAsync(id) != null;
}
```

```
// IMPORTANT: default methods are only accessible through the interface reference!
var repo = new SqlOrderRepository(context);
repo.ExistsAsync(1); // compile error if SqlOrderRepository didn't override

IOrderRepository irepo = repo;
irepo.ExistsAsync(1); // accessible through interface reference
```

Interfaces in DI

```
// Define contracts:
public interface IOrderService { Task<Order?> GetByIdAsync(int id); }

// Consume contracts -- never concrete types:
public class CheckoutService(IOrderService orders, IEmailService email) { ... }

// Register in Program.cs:
builder.Services.AddScoped<IOrderService, OrderService>();
builder.Services.AddScoped<IEmailService, SmtEmailService>();
```

Key Takeaways

- An interface is a pure contract — no state, no constructor, just signatures
- A class can implement many interfaces but inherit only one class
- Dependency injection in .NET is built on interfaces
- Default interface methods (C# 8+) allow interface evolution — but only accessible through interface reference

	Interface	Abstract Class
Fields	No	Yes
Constructor	No	Yes (protected)
Default impl	Yes (C# 8+)	Yes (full)
Multiple per class	Yes	No (one only)
Access modifiers	No (pub implicit)	Yes

.NET-specific rules:

- Interface names start with I by convention — always (IOrderService, IRepository)
- Register interfaces in DI with AddScoped, AddTransient, or AddSingleton
- IDisposable — implement whenever your class holds unmanaged resources

Memory aid: An interface is a job description. It says what you must be able to do. A class is the employee who fulfils that description — and one employee can hold multiple job titles.

Debugging Tips

Symptom	Cause	Fix
does not implement interface member	Missing one or more interface method implementations	Implement all members — IDE will offer to generate stubs
Default interface method not found on concrete type	Default methods only accessible via interface reference, not concrete type	Cast to interface type or override the method in the class

Symptom	Cause	Fix
DI throws Unable to resolve service for type IXxx	Interface not registered in Program.cs	Add builder.Services.AddScoped()
Explicit implementation not accessible	Explicit implementations are hidden from the concrete type	Access through an interface variable: IFoo f = obj; f.Method()
Mock in tests doesn't intercept calls	Test is creating the concrete class directly	Inject the mock through the interface

Self-check — you should now be able to:

- Write an interface and two implementations that can be swapped via DI
- Explain why interfaces are central to testability and dependency injection
- Explain what default interface methods are, their key limitation, and when to use them

2.10. Sealed Classes and Methods

Builds on: 2.6 (inheritance — sealed stops it), 2.7 (polymorphism — sealed stops override chains), 2.8 (abstract classes — the opposite end of the spectrum).

What does sealed do and why does it exist?

sealed prevents inheritance or further overriding. Design intent: some classes represent a complete, specific concept with no meaningful variations. Performance: the JIT can optimise calls to sealed methods more aggressively.

```
public sealed class SqlConnectionString
{
    public string Server { get; }
    public int Port { get; }
    public bool Encrypt { get; }

    public SqlConnectionString(string server, int port = 1433, bool encrypt = true)
    { Server = server; Port = port; Encrypt = encrypt; }

    public override string ToString()
        => $"Server={Server},{Port};Encrypt={Encrypt}";
}

public class Custom : SqlConnectionString { }    // compile error
```

Sealed methods and sealed record hierarchies

```
// sealed override -- prevents further overriding:
public class CreditCardPayment : Payment
{
    public sealed override string GetReceipt()
        => $"Credit card payment: {Amount:C}";
}

// Sealed record hierarchies -- exhaustive switch:
public abstract record Shape;
```

```

public sealed record Circle(double Radius) : Shape;
public sealed record Rectangle(double Width, double Height) : Shape;

double area = shape switch
{
    Circle c    => Math.PI * c.Radius * c.Radius,
    Rectangle r => r.Width * r.Height,
    // Compiler verifies exhaustiveness
};

```

Singleton pattern

```

public sealed class Singleton
{
    private Singleton() { }
    public static Singleton Instance { get; } = new Singleton();
}

```

Key Takeaways

- sealed on a class prevents any inheritance
- sealed override on a method prevents further overriding of that specific method
- Structs are implicitly sealed — you cannot inherit from a struct
- sealed communicates design intent: this type is not designed to be extended
- Value objects, identifiers, and leaf exception types are natural candidates for sealed

Memory aid: sealed is a dead-end sign on a road. You can drive up to it, but you cannot continue past it.

Debugging Tips

Symptom	Cause	Fix
cannot derive from sealed type	Trying to inherit a sealed class	Remove inheritance, use composition instead
cannot override sealed member	Trying to override a sealed override — it is final	Remove the override — the sealed version is final
Struct inheritance attempt fails	Structs are implicitly sealed	Use a class or interface instead

Self-check — you should now be able to:

- Apply sealed to a class and explain what it prevents
- Explain why structs are always sealed
- Use sealed record hierarchies with switch expressions for exhaustive pattern matching

2.11. Static Classes and Members

Builds on: 2.1 (classes), 2.2 (constructors — static constructors), 1.18 (const/readonly — static readonly), 1.19 (scope — static fields live for the entire app lifetime).

What are static classes and members and why do they exist?

Static members belong to the type itself, not any instance. Static classes exist for stateless, instance-independent scenarios: mathematical operations, pure utility functions, factory methods, extension methods.

```
public static class MathHelpers
{
    public static decimal RoundToNearest(decimal value, decimal nearest)
        => Math.Round(value / nearest) * nearest;

    public static decimal Clamp(decimal value, decimal min, decimal max)
        => Math.Max(min, Math.Min(max, value));
}
```

Extension methods

```
public static class StringExtensions
{
    public static bool IsValidEmail(this string value)
        => !string.IsNullOrEmpty(value) && value.Contains('@') && value.Contains('.');

    public static string Truncate(this string value, int maxLength, string suffix = "...")
    {
        if (string.IsNullOrEmpty(value) || value.Length <= maxLength) return value;
        return value[..maxLength] + suffix;
    }
}

"s@zoho.com".IsValidEmail()    // true
"Hello World".Truncate(5)      // "Hello..."
```

IQueryable extension methods for EF Core

```
public static class OrderQueryExtensions
{
    public static IQueryable<Order> Active(this IQueryable<Order> query)
        => query.Where(o => o.Status != "Cancelled");

    public static IQueryable<Order> ForUser(this IQueryable<Order> query, int userId)
        => query.Where(o => o.UserId == userId);

    public static IQueryable<Order> Paginate(
        this IQueryable<Order> query, int page, int pageSize)
        => query.Skip((page - 1) * pageSize).Take(pageSize);
}

var orders = await _context.Orders
    .Active().ForUser(userId).Paginate(page, pageSize).ToListAsync();
```

Warning — static mutable state in ASP.NET Core

```
// DANGEROUS -- shared across all concurrent requests:
private static List<Order> _pendingOrders = new();

// SAFE -- instance field, each request gets its own service:
```

```
private List<Order> _pendingOrders = new();
```

Key Takeaways

- Static members belong to the type, not any instance — one shared copy for the entire application
- Static classes are sealed, cannot be instantiated, and can only contain static members
- Extension methods must be in static classes
- Static fields in ASP.NET Core are shared across all concurrent requests — use carefully

Memory aid: Static = carved in stone on the building itself. Instance = written on sticky notes attached to each tenant.

Debugging Tips

Symptom	Cause	Fix
Extension method not found	Namespace of static class not imported	Add using for the namespace containing the extension class
Static field causes data bleed between requests	Mutable static state shared across all requests in ASP.NET Core	Move to instance field or use DI Scoped/Transient lifetime
Race condition on static counter	Unsynchronised ++ on static field from multiple threads	Use Interlocked.Increment
cannot declare instance members in a static class	Trying to add an instance field or method to a static class	Move the instance member to a regular class
HttpClient exhausting sockets	Creating new HttpClient() per request	Use static readonly HttpClient or IHttpClientFactory

Self-check — you should now be able to:

- Write a static class with extension methods and use them via dot notation
- Explain why mutable static state is dangerous in ASP.NET Core
- Identify where extension methods are used in everyday .NET code

2.12. Partial Classes

Builds on: 2.1 (classes), 1.2 (project structure), 2.11 (static — code generators use partial classes extensively).

What are partial classes and why do they exist?

A partial class splits a single class definition across multiple files. At compile time, all parts are merged into one class. Primary use: code generation — tools like EF Core migrations, scaffolding, and Blazor generate code automatically. Partial classes let the generator own one file and you own another.

```
// Order.cs -- your business logic:
public partial class Order
{
```

```

    public int Id { get; private set; }
    public string Status { get; private set; } = "Pending";
    public void Cancel() { Status = "Cancelled"; }
}

// Order.Validation.cs -- separated by concern:
public partial class Order
{
    public bool IsValid() => Status != null;
    public IEnumerable<string> GetValidationErrors()
    {
        if (Status == null) yield return "Status is required";
    }
}

```

Partial methods — the handshake pattern

```

// Generated part:
public partial class Order
{
    partial void OnStatusChanged(string oldStatus, string newStatus);

    public void SetStatus(string newStatus)
    {
        var old = Status;
        Status = newStatus;
        OnStatusChanged(old, newStatus); // silently removed if not implemented
    }
}

// Your part -- optionally implement:
public partial class Order
{
    partial void OnStatusChanged(string oldStatus, string newStatus)
        => Console.WriteLine($"{Id}: {oldStatus} -> {newStatus}");
}

```

Key Takeaways

- Partial classes split one class definition across multiple files — compiled into one class
- The primary purpose is code generation — separating generated code from hand-written code
- All parts must be in the same namespace and assembly, and must agree on access modifiers
- Partial methods are optional hooks — declared in one part, optionally implemented in another
- Never edit generated partial class files — they'll be overwritten

Memory aid: Partial classes are like a jigsaw puzzle. Each file is one piece. The compiler assembles them at build time.

Debugging Tips

Symptom	Cause	Fix
partial modifier syntax error	partial must directly precede class — public partial class not partial public class	Write public partial class, not partial public class
Member from one partial part not found in another	Namespace mismatch between files	Ensure both files declare the same namespace
Generated file overwrites your changes	You edited a *.Designer.cs or generated file directly	Move your code to the hand-written partial — never edit generated files
Partial method implementation not compiling	Return type or access modifier on pre-C# 9 partial method	Make it void and private for pre-C# 9, or upgrade to C# 9+ syntax
Two partial class files disagree on base class	Each specifies a different base	Pick one file to declare the base class — leave the other without it

Self-check — you should now be able to:

- Explain what a partial class is and the compiler-level mechanism behind it
- Explain partial methods — declaration, optional implementation, and what happens when not implemented
- Explain why you should never edit generated partial class files

2.13. Records (record class, record struct, with expressions)

Builds on: 2.1 (classes — records are a class variant), 2.3 (properties — records auto-generate init-only properties), 1.16 (structs), 1.22 (tuples).

What are records and why do they exist?

Before records, modelling immutable data required significant boilerplate — constructors, Equals, GetHashCode, ToString, all written manually. Records were introduced in C# 9 for immutable data modelling: DTOs, value objects, API responses, domain events.

```
// One line replaces all the boilerplate:
public record OrderSummary(int Id, string CustomerName, decimal Amount);

// Auto-generates: constructor, init-only properties, Equals, GetHashCode,
// ToString, Deconstruct, and copy constructor for with expressions.

var a = new OrderSummary(1, "Sathish", 999.99m);
var b = new OrderSummary(1, "Sathish", 999.99m);

Console.WriteLine(a == b); // true -- value-based equality
Console.WriteLine(a);      // OrderSummary { Id = 1, CustomerName = Sathish, ... }

var (id, name, amount) = a; // deconstruction works automatically
```

with expressions — non-destructive mutation

```
public record Order(int Id, string Status, decimal Amount, string CustomerName);

var original = new Order(1, "Pending", 999.99m, "Sathish");
```

```

var shipped = original with { Status = "Shipped" };
var amended = original with { Status = "Processing", Amount = 1099.99m };

Console.WriteLine(original.Status);    // "Pending" -- unchanged
Console.WriteLine(shipped.Status);    // "Shipped"
Console.WriteLine(amended.Amount);    // 1099.99

```

Compact constructor for validation

```

public record Money(decimal Amount, string Currency)
{
    public Money
    {
        if (Amount < 0) throw new ArgumentException("Amount cannot be negative");
        if (string.IsNullOrEmpty(Currency)) throw new ArgumentException("Currency required");
        Currency = Currency.ToUpperInvariant();
    }

    public Money Add(Money other)
    {
        if (Currency != other.Currency) throw new InvalidOperationException("Currency mismatch");
        return this with { Amount = Amount + other.Amount };
    }
}

```

record struct and readonly record struct

```

public record struct Point(int X, int Y);
public readonly record struct Money(decimal Amount, string Currency);

var p1 = new Point(1, 2);
var p2 = new Point(1, 2);
Console.WriteLine(p1 == p2);    // true -- value equality
Console.WriteLine(p1);          // "Point { X = 1, Y = 2 }"

var p3 = p1 with { X = 99 };    // with works on record struct too

```

Sealed record hierarchies

```

public abstract record DomainEvent(DateTime OccurredAt);
public sealed record OrderPlaced(int OrderId, string CustomerEmail,
    decimal Amount, DateTime OccurredAt) : DomainEvent(OccurredAt);
public sealed record OrderShipped(int OrderId, string TrackingNumber,
    DateTime OccurredAt) : DomainEvent(OccurredAt);

public async Task HandleEventAsync(DomainEvent evt)
{
    await (evt switch
    {
        OrderPlaced e    => SendConfirmationEmailAsync(e.CustomerEmail, e.Amount),
        OrderShipped e    => SendShippingNotificationAsync(e.OrderId, e.TrackingNumber),
        _                 => Task.CompletedTask
    });
}

```

Key Takeaways

- Records are for immutable data modelling — value-based equality, clean syntax
- record class — reference type, structural equality, immutable by default
- record struct — value type, same features but stack-allocated
- with expressions create a NEW record with specified properties changed — original unchanged
- Compact constructors allow validation without repeating parameter lists
- EF Core entities should be classes, not records — EF needs mutable state

	record class	record struct	readonly record struct
-----+-----+-----+-----			
Type	Reference	Value	Value
Equality	Structural	Structural	Structural
Mutable by default	No	Yes	No
Inheritance	Yes	No	No
with support	Yes	Yes	Yes

Memory aid: A record is a snapshot — a photo of data at a point in time. You don't edit the photo. If things change, you take a new photo (with).

Debugging Tips

Symptom	Cause	Fix
Record properties can't be set after construction	init-only — intentional	Use with to create a modified copy, or use { get; set; } if mutation is needed
Two records not equal despite same values	One is a derived type of the other — records include type in equality	Ensure both are the exact same record type
EF Core fails to track record entity	Records are immutable — EF change tracking requires mutable state	Use a class for EF entities
with expression gives compile error	with only works on records and structs	Convert to a record or write a manual copy method
JSON deserialization produces null properties	Positional record constructor parameters don't match JSON property names	Use [JsonPropertyName] attributes or a nominal record with { get; set; }

Self-check — you should now be able to:

- Create a positional record and explain every member it auto-generates
- Use a with expression to create a modified copy
- Explain the difference between record, record struct, and readonly record struct

2.14. Object Initializers

Builds on: 2.1, 2.2, 2.3, 2.13.

What are object initializers and why do they exist?

Object initializers collapse multiple property assignment statements into a single expression — cleaner, composable, and usable in contexts where separate statements aren't allowed (LINQ projections, return statements).

```
// Old way:
var order = new Order();
order.Id = 1; order.CustomerName = "Sathish"; order.Amount = 999.99m;

// With object initializer:
var order = new Order { Id = 1, CustomerName = "Sathish", Amount = 999.99m };
```

How they work and what properties can be set

```
// Compiler translates to:
var __temp = new Order();
__temp.Id = 1; __temp.Amount = 500m;
var order = __temp;    // constructor runs FIRST

// Only accessible setters work:
public int Id { get; set; }           // works
public string Email { get; init; }    // works in initializer
public string Role { get; private set; } // compile error -- not accessible
public string Hash { get; }           // compile error -- no setter
public int Age => 30;                  // compile error -- computed
```

Nested, collection, and required

```
// Nested:
var order = new Order
{
    Id = 1,
    ShippingAddress = new Address { Street = "42 MG Road", City = "Chennai" },
    Items = new List<OrderItem>
    {
        new() { ProductId = 1, Quantity = 2, Price = 99.99m },
    }
};

// Required properties -- C# 11:
public class CreateOrderDto
{
    public required string CustomerName { get; init; }
    public required decimal Amount { get; init; }
    public string? Notes { get; init; }           // optional
}

// Target-typed new -- C# 9:
Order order = new() { Id = 1, Amount = 500m };
var orders = new List<Order> { new() { Id = 1 }, new() { Id = 2 } };
```

Object initializers in LINQ

```
var orderDtos = orders
    .Where(o => o.IsActive)
    .Select(o => new OrderDto
```

```

{
    Id = o.Id, CustomerName = o.CustomerName,
    Amount = o.Amount, Status = o.Status
})
.ToList();

```

Key Takeaways

- Object initializers set properties after the constructor runs — in a single expression
- Only properties with accessible setters (set or init) can be initialized this way
- init-only properties can be set in an object initializer but not after construction
- required properties (C# 11) must be set in the initializer — compile error if omitted
- Target-typed new() omits the type name when it's inferrable from context
- Object initializers are essential in LINQ projections and test data setup

Memory aid: An object initializer is like filling out a form immediately after it's handed to you. The form (constructor) gives you the blank template; you fill in the fields before you hand it back.

Debugging Tips

Symptom	Cause	Fix
Property set in initializer has no effect	Property has { get; } only — no setter	Add init or set accessor
init-only property set outside initializer	Trying to set an init property in a method after construction	Use with (for records) or redesign to set it at construction
required property error	Forgot to set a required property in object initializer	Set all required properties or mark them optional
Nested object is null after initialization	Forgot new Type {} — omitted the property	Add the nested object initializer
JSON deserialization gives null for all properties	Class has only constructor and { get; } properties	Add { get; set; } or { get; init; } and ensure property names match JSON

Self-check — you should now be able to:

- Explain what an object initializer is and what the compiler generates from it
- Use nested object initializers and collection initializers to build complex object graphs
- Explain why object initializers are essential in LINQ Select projections

2.15. The this Keyword

Builds on: 2.1, 2.2, 2.3, 2.11 (extension methods — this marks the extended type).

What is this and why does it exist?

this is a reference to the current instance. It exists to: disambiguate when a parameter shadows a field, chain constructors, pass the current object to another method or return it, and mark extension method parameters.

```
// Disambiguation (usually avoid with _ prefix convention):
this.name = name;

// Constructor chaining:
public Order(int id, string name) : this(id, name, 0m) { }

// Pass current object:
registry.Register(this);

// Fluent builder -- return this:
public QueryBuilder From(string table) { _table = table; return this; }
public QueryBuilder Where(string cond) { _conditions.Add(cond); return this; }

var query = new QueryBuilder().From("Orders").Where("Status = 'Pending']").Build();

// Extension method marker:
public static bool IsHighValue(this Order order) => order.Amount > 10000;

// Indexer:
public Order? this[int id] { get => _orders.TryGetValue(id, out var o) ? o : null; }
```

When you actually need this in modern C#

```
// NEEDED -- constructor chaining:
public Order(int id, string name) : this(id, name, 0m) { }

// NEEDED -- passing current object or fluent return:
registry.Register(this);
return this;

// NEEDED -- extension method declaration:
public static bool IsValid(this Order order) { }

// OPTIONAL -- disambiguation (use _ prefix instead):
this.name = name;    // prefer: _name = name

// NOT NEEDED:
this.Status = "Pending";    // just write Status = "Pending"
```

Key Takeaways

- this is a reference to the current instance
- In modern C# with _ prefix convention, disambiguation is rarely needed
- Constructor chaining: : this(args) — chained runs first
- Fluent builders: return this — enables method chaining
- Extension methods: this T param — marks the type being extended
- Indexers: public T this[int index] — enables [] access on custom types

Memory aid: this is your nametag at a conference. Most of the time you don't need to look at it — context makes it clear. But when two people have the same name (shadowed field), you point to your nametag.

Debugging Tips

Symptom	Cause	Fix
Field always has default value despite constructor assignment	name = name assigns parameter to itself — self-assignment bug	Add this. prefix or use the _ naming convention
Constructor chaining causes infinite loop	Constructor calls itself instead of a different overload	Ensure : this(args) calls a different overload, not the same signature
Fluent method not chainable	Method returns void instead of this	Change return type to the class type and add return this
Extension method compile error	this missing from first parameter	Add this before the first parameter type
Indexer not found	Class doesn't implement this[key] indexer	Add public T this[KeyType key] getter/setter

Self-check — you should now be able to:

- Explain what this refers to and in which situations you genuinely need to write it
- Write constructor chaining with : this(args) and explain execution order
- Implement a fluent builder using return this for method chaining

2.16. Method Hiding vs Overriding (new vs override)

Builds on: 2.6 (inheritance), 2.7 (polymorphism — overriding enables it, hiding deliberately breaks it), 2.8 (abstract classes — abstract methods require override, never new).

What is the difference and why does it matter?

- override replaces the base method. Polymorphism works — calls through a base reference invoke the derived version.
- new hides the base method. Polymorphism is broken — calls through a base reference still invoke the base version.

```
public class Animal { public virtual string Speak() => "..."; }

// Dog uses override:
public class Dog : Animal { public override string Speak() => "Woof"; }

// Cat uses new:
public class Cat : Animal { public new string Speak() => "Meow"; }

Animal dog = new Dog();
Animal cat = new Cat();

Console.WriteLine(dog.Speak()); // "Woof" -- override, derived version runs
Console.WriteLine(cat.Speak()); // "..." -- new/hiding, base version runs!
```

The realistic bug scenario

```
public abstract class BaseValidator
{
    public virtual bool Validate(object input) { return input != null; }
```

```

}

public class OrderValidator : BaseValidator
{
    // Bug: uses new instead of override:
    public new bool Validate(object input)
        => base.Validate(input) && (input is Order o && o.Amount > 0);
}

// In the pipeline -- OrderValidator's logic NEVER runs:
BaseValidator validator = new OrderValidator();
validator.Validate(order);    // base class version runs!

// Fix:
public class OrderValidator : BaseValidator
{
    public override bool Validate(object input)    // polymorphism now works
        => base.Validate(input) && (input is Order o && o.Amount > 0);
}

```

Key Takeaways

- **override** — replaces the base method. Runtime uses actual object type. Polymorphism works.
- **new** — hides the base method. Runtime uses variable's declared type. Polymorphism breaks.
- The difference only matters when calling through a base class or interface reference
- Omitting both keywords compiles with a warning — behaves like **new**
- **new** is almost always wrong in application code

	override	new
-----+-----+-----		
Base must	virtual or abstract	Any (virtual or not)
Dispatch	Actual object type	Declared variable type
Polymorphism	Works correctly	Broken
Warning	Yes if omitted	Silenced by adding new
Use when	Almost always	Can't modify base class

Memory aid: **override** = upgrade the engine in the same car body. **new** = build a different car with the same name. People asking for 'a car' still get the old model.

Debugging Tips

Symptom	Cause	Fix
Derived class method never called despite being registered	Method uses new instead of override — base method runs through base reference	Change new to override , ensure base method is virtual
Compiler warning: hides inherited member	Method shadows base without new keyword	Add override (preferred) or new (if hiding is intentional)
Cannot use override	Base method is not virtual or abstract	Add virtual to base method, or accept that hiding is the only option

Symptom	Cause	Fix
base.Method() in hiding method calls wrong version	base. always resolves to declared base at compile time — this is expected	This is correct behaviour — base. bypasses virtual dispatch
Mock doesn't intercept calls	Method not virtual — mocking frameworks require virtual or abstract	Make method virtual or abstract to enable mocking

Self-check — you should now be able to:

- Explain the runtime difference between override and new with a concrete code example
- Identify the narrow legitimate cases where new is appropriate
- Debug a scenario where a derived class method is never called because it uses new instead of override

2.17. Object Equality — Equals, GetHashCode, == Overloading

Builds on: 2.1 (classes), 2.13 (records — records auto-generate correct equality), 1.4 (value vs reference types), 2.7 (polymorphism — Equals is a virtual method on object).

What is equality in C# and why is it complex?

Equality in C# is a contract between three mechanisms that must stay consistent: Equals(object obj), GetHashCode(), and the == operator. By default, classes use reference equality — two variables are equal only if they point to the same object. This is usually wrong for domain objects and DTOs.

The equality contract — four rules

- Reflexive: x.Equals(x) must return true
- Symmetric: if x.Equals(y) is true, then y.Equals(x) must be true
- Transitive: if x.Equals(y) and y.Equals(z), then x.Equals(z) must be true
- Consistent with GetHashCode: if x.Equals(y) is true, x.GetHashCode() == y.GetHashCode() must be true

Breaking rule 4 causes silent, catastrophic bugs in dictionaries and hash sets.

The dictionary bug from missing GetHashCode

```
// Broken -- Equals overridden but GetHashCode not:
public class OrderId
{
    public int Value { get; }
    public override bool Equals(object? obj)
        => obj is OrderId other && Value == other.Value;
    // GetHashCode NOT overridden!
}

var dict = new Dictionary<OrderId, string>();
```

```

var id1 = new OrderId(42);
dict[id1] = "Order A";

var id2 = new OrderId(42);    // equal to id1 by Equals
dict.ContainsKey(id2);        // FALSE -- different hash code, wrong bucket!
dict[id2];                    // KeyNotFoundException

```

Full implementation

```

public sealed class Money : IEquatable<Money>
{
    public decimal Amount { get; }
    public string Currency { get; }

    public Money(decimal amount, string currency)
    {
        if (amount < 0) throw new ArgumentException("Amount cannot be negative");
        Amount = amount;
        Currency = currency?.ToUpperInvariant()
            ?? throw new ArgumentNullException(nameof(currency));
    }

    public bool Equals(Money? other)
    {
        if (other is null) return false;
        if (ReferenceEquals(this, other)) return true;
        return Amount == other.Amount && Currency == other.Currency;
    }

    public override bool Equals(object? obj) => Equals(obj as Money);

    public override int GetHashCode() => HashCode.Combine(Amount, Currency);

    public static bool operator ==(Money? left, Money? right)
    {
        if (left is null && right is null) return true;
        if (left is null || right is null) return false;
        return left.Equals(right);
    }

    public static bool operator !=(Money? left, Money? right) => !(left == right);

    public override string ToString() => $"{Amount:F2} {Currency}";
}

```

Records — equality is already done correctly

```

public record OrderId(int Value);

var a = new OrderId(42);
var b = new OrderId(42);
Console.WriteLine(a == b);           // true -- generated
Console.WriteLine(a.Equals(b));       // true -- generated
Console.WriteLine(a.GetHashCode() == b.GetHashCode()); // true -- generated

```

If your type is primarily a data carrier with no mutable state, use a record — equality is generated correctly and automatically.

Key Takeaways

- Default class equality is reference-based — two objects with identical data are not equal
- Overriding Equals without overriding GetHashCode causes catastrophic bugs in collections
- If `x.Equals(y)` is true, `x.GetHashCode()` must equal `y.GetHashCode()` — non-negotiable
- Overriding Equals does NOT change `==` — you must overload `==` separately
- Always overload `==` and `!=` together
- Use records for value objects — equality is generated correctly and automatically

Step	Method	Required?
1	override <code>bool Equals()</code>	Yes
2	override <code>int GetHashCode()</code>	Yes -- always with Equals
3	<code>bool Equals(T?) IEquat</code>	Yes -- strongly recommended
4	operator <code>==(T?, T?)</code>	Yes if <code>==</code> should be structural
5	operator <code>!=(T?, T?)</code>	Yes -- always with <code>==</code>

.NET-specific rules:

- `HashCode.Combine(field1, field2, ...)` — use this, not manual XOR
- Records auto-generate all five — prefer records for value objects
- EF Core change tracking depends on correct GetHashCode
- Never compute GetHashCode from mutable fields when object is in a hash-based collection

Memory aid: Equals and GetHashCode are partners. If Equals says two objects are twins, GetHashCode must give them the same fingerprint. A twin with a different fingerprint will get lost in any dictionary or HashSet.

Debugging Tips

Symptom	Cause	Fix
Dictionary.ContainsKey returns false for a key that was added	Equals overridden but GetHashCode not — key ends up in different bucket	Override GetHashCode to be consistent with Equals
HashSet contains duplicates that should be equal	Same cause — GetHashCode inconsistency	Override GetHashCode alongside Equals
<code>a.Equals(b)</code> true but <code>a == b</code> false	Equals overridden but <code>==</code> not overloaded	Overload <code>==</code> and <code>!=</code> operators
LINQ Distinct() doesn't remove duplicates	Missing or broken Equals/GetHashCode	Implement IEquatable and override GetHashCode correctly
EF Core doesn't detect entity modification	GetHashCode is unstable — returns different values for the same object over time	Base GetHashCode only on immutable fields

Self-check — you should now be able to:

- Explain why overriding Equals without GetHashCode is dangerous and demonstrate the bug

- Implement the complete equality pattern (Equals, GetHashCode, IEquatable, ==, !=) for a class
- Explain why records are preferred for value objects — what they auto-generate and why it matters

Section 2 complete. All 17 topics from 2.1 through 2.17 are covered. Say next to begin Section 3: Generics.